

东北大学资源与土木工程学院

作 业 报 告

课程设计名称: 空间数据库原理

学 生 专 业: 遥感科学与技术 学生班级: 遥感 2301 班

学 号 : 20232411 学生姓名: 张洋

指导教师姓名: 郭甲腾

指导教师评估:

评 阅 人 :

评阅时间:

空间存储和索引作业

一、计算(1, 0), (3, 8), (4, 6), (7, 9)坐标点的 Z 曲线索引值。

1. 计算坐标(1,0):

X 坐标 1 转化为二进制结果为: 0001; Y 坐标 0 转换为二进制结果为: 0000;
隔行扫描二进制数字的比特到一个字符串结果为: 00000010; Z 索引值为: 2。

2. 计算坐标(3,8):

X 坐标 3 转化为二进制结果为: 0011; Y 坐标 8 转换为二进制结果为: 1000;
隔行扫描二进制数字的比特到一个字符串结果为: 01001010; Z 索引值为: 74。

3. 计算坐标(4,6):

X 坐标 4 转化为二进制结果为: 0100; Y 坐标 0 转换为二进制结果为: 0110;
隔行扫描二进制数字的比特到一个字符串结果为: 00110100; Z 索引值为: 52。

4. 计算坐标(7,9):

X 坐标 1 转化为二进制结果为: 0111; Y 坐标 0 转换为二进制结果为: 1001;
隔行扫描二进制数字的比特到一个字符串结果为: 01101011; Z 索引值为: 107。

Python:

```
def z_index(x, y):  
    print("====计算坐标({},{}),====".format(x,y))  
    x_bin = bin(x)[2:].zfill(4)  
    y_bin = bin(y)[2:].zfill(4)  
  
    index = ''.join([a + b for a, b in zip(x_bin, y_bin)])  
    index_ten = int(index, 2)  
    print("x->{}, y->{}, 合并->{}, Z 索引值->{}".format(x_bin,y_bin,index,index_ten))  
  
coordinates = [(1,0), (3, 8), (4, 6), (7, 9)]  
for x, y in coordinates:  
    index = z_index(x, y)
```

运行结果:

```
====计算坐标(1,0)====  
x->0001, y->0000, 合并->00000010, Z索引值->2  
====计算坐标(3,8)====  
x->0011, y->1000, 合并->01001010, Z索引值->74  
====计算坐标(4,6)====  
x->0100, y->0110, 合并->00110100, Z索引值->52  
====计算坐标(7,9)====  
x->0111, y->1001, 合并->01101011, Z索引值->107
```

二、请计算并绘制下图的 3 阶 Hilbert 曲线。

读入 x 和 y 坐标的 3 比特二进制表示。隔行扫描二进制比特到一个字符串。将字符串自左至右分成 2 比特长的串。规定每个 2 比特长的串的十进制值，如“00”等于 0，“01”等于 1，“10”等于 3，“11”等于 2。

对于数组中每个数字 j ，如果 $j=0$ 把后面数组中出现的所有 1 变成 3，并把所有出现的 3 变成 1。 $j=3$ 把后面数组中出现的所有 0 变成 2，并把所有出现的 2 变成 0。

将数组中每个值按步骤转换成二进制表示，自左至右连接所有的串，并计算其十进制值。

010101	010111	011101	011111	110101	110111	111101	111111
010100	010110	011100	011110	110100	110110	111100	111110
010001	010011	011001	011011	110001	110011	111001	111011
010000	010010	011000	011010	110000	110010	111000	111010
000101	000111	001101	001111	100101	100111	101101	101111
000100	000110	001100	001110	100100	100110	101100	101110
000001	000011	001001	001011	100001	100011	101001	101011
000000	000010	001000	001010	100000	100010	101000	101010

111	112	121	122	211	212	221	222
110	113	120	123	210	213	220	223
103	102	131	130	203	202	231	230
100	101	132	133	200	201	232	233
033	030	023	022	311	310	303	300
032	031	020	021	312	313	302	301
001	002	013	012	321	320	331	332
000	003	010	011	322	323	330	333

21	22	25	26	37	38	41	42
20	23	24	27	36	39	40	43
19	18	29	28	35	34	45	44
16	17	30	34	32	33	46	47
15	12	11	10	53	52	51	48
14	13	8	9	54	55	50	49
1	2	7	6	57	56	61	62
0	3	4	5	58	59	60	63

Python:

```
import numpy as np
def tdimh(a, b):
    a_bin = format(a, '03b')
    b_bin = format(b, '03b')
    c = a_bin[0] + b_bin[0] + a_bin[1] + b_bin[1] + a_bin[2] + b_bin[2]

    d1 = binh(a_bin[0] + b_bin[0])
    d2 = binh(a_bin[1] + b_bin[1])
    d3 = binh(a_bin[2] + b_bin[2])

    s = [d1, d2, d3]

    if d1 == 0:
        if d2 == 3:
            d2 = 1
        elif d2 == 1:
            d2 = 3

        if d3 == 3:
            d3 = 1
        elif d3 == 1:
            d3 = 3

    if d1 == 3:
        if d2 == 0:
            d2 = 2
        elif d2 == 2:
            d2 = 0

        if d3 == 0:
            d3 = 2
        elif d3 == 2:
            d3 = 0

    if d2 == 0:
        if d3 == 1:
            d3 = 3
        elif d3 == 3:
            d3 = 1

    if d2 == 3:
        if d3 == 0:
            d3 = 2
```

```

        elif d3 == 2:
            d3 = 0

    s = [d1, d2, d3]
    o = d1 * 4**2 + d2 * 4 + d3
    return c, s, o
def binh(y):
    if y == '00':
        return 0
    elif y == '01':
        return 1
    elif y == '10':
        return 3
    elif y == '11':
        return 2
    else:
        return None
x = np.array([0, 1, 2, 3, 4, 5, 6, 7])
y = np.array([0, 1, 2, 3, 4, 5, 6, 7])
s = np.zeros((8, 8), dtype=int)
c_matrix = np.empty((8, 8), dtype=object)
m_matrix = np.empty((8, 8), dtype=object)
for i in range(8):
    for j in range(8):
        c, m, o = tdimh(x[i], y[j])
        s[j, i] = o
        c_matrix[j, i] = c
        m_matrix[j, i] = ''.join(map(str, m))
op = np.zeros((8, 8), dtype=int)
for k in range(8):
    opp = s[8 - k - 1, :]
    op[k, :] = opp
c_matrix_flipped = np.empty((8, 8), dtype=object)
for k in range(8):
    c_matrix_flipped[k, :] = c_matrix[8 - k - 1, :]
m_matrix_flipped = np.empty((8, 8), dtype=object)
for k in range(8):
    m_matrix_flipped[k, :] = m_matrix[8 - k - 1, :]
# 打印结果
print("\n 转换后的六位二进制矩阵:")
print(c_matrix_flipped)
print("\n 转换的 3bit 数据矩阵:")
print(m_matrix_flipped)
print("\n 最终输出矩阵:")

```

```

print(op)
def transformMatrix(s):
    transformed_s = s.copy()
    for i in range(len(s)):
        if s[i] == 0:
            transformed_s[i+1:] = 4 - transformed_s[i+1:]
        elif s[i] == 3:
            transformed_s[i+1:] = 2 - transformed_s[i+1:]
    return transformed_s

```

运行结果:

转换后的六位二进制矩阵:

```

[['010101' '010111' '011101' '011111' '110101' '110111' '111101' '111111']
 ['010100' '010110' '011100' '011110' '110100' '110110' '111100' '111110']
 ['010001' '010011' '011001' '011011' '110001' '110011' '111001' '111011']
 ['010000' '010010' '011000' '011010' '110000' '110010' '111000' '111010']
 ['000101' '000111' '001101' '001111' '100101' '100111' '101101' '101111']
 ['000100' '000110' '001100' '001110' '100100' '100110' '101100' '101110']
 ['000001' '000011' '001001' '001011' '100001' '100011' '101001' '101011']
 ['000000' '000010' '001000' '001010' '100000' '100010' '101000' '101010']]

```

转换的3bit数据矩阵:

```

[['111' '112' '121' '122' '211' '212' '221' '222']
 ['110' '113' '120' '123' '210' '213' '220' '223']
 ['103' '102' '131' '130' '203' '202' '231' '230']
 ['100' '101' '132' '133' '200' '201' '232' '233']
 ['033' '030' '023' '022' '311' '310' '303' '300']
 ['032' '031' '020' '021' '312' '313' '302' '301']
 ['001' '002' '013' '012' '321' '320' '331' '332']
 ['000' '003' '010' '011' '322' '323' '330' '333']]

```

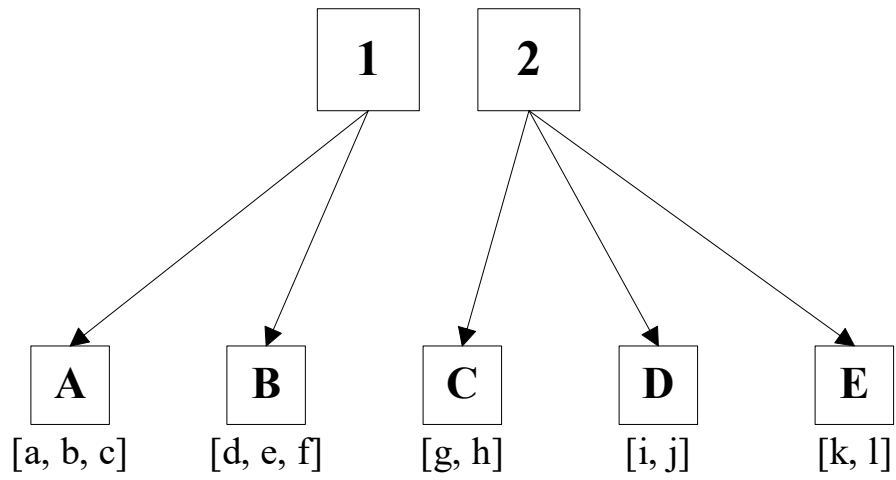
最终输出矩阵:

```

[[21 22 25 26 37 38 41 42]
 [20 23 24 27 36 39 40 43]
 [19 18 29 28 35 34 45 44]
 [16 17 30 31 32 33 46 47]
 [15 12 11 10 53 52 51 48]
 [14 13 8 9 54 55 50 49]
 [1 2 7 6 57 56 61 62]
 [0 3 4 5 58 59 60 63]]

```

三、根据下图画出相应的 R 树，并写出虚线搜索框所要搜索的节点。叶节点为 a, b, ..., l.



虚线框所要搜索的节点为：节点 B 下的 d、e、节点 C 下的 g、节点 D 下的 i。